

Graph Cloning / State Graph Dijkstra Master Notes

Source chat: [ChatGPT Graph Cloning Task](#)

Important:

- These notes are prepared from the linked ChatGPT conversation and the code patterns discussed there.
- Full external problem statements are rewritten in original words for notes/practice.
- Main revision question for every problem: **Can reaching the same node with different extra information change the future?**

Core Pattern

State Rule

If normal state is:

(node)

and some extra information affects future choices, convert it into:

(node, extraInfo)

Examples:

Situation	State
Stops limit	(node, stopsUsed)
Coupon / discount	(node, couponUsed)
K free hops	(node, hopsUsed)
Switch mode	(node, switchState)
Graph orientation	(node, graphMode)
Previous edge matters	(node, lastEdgeWeight)
Multiple one-time operations	(node, mask)

Algorithm Choice

Edge Cost Type	Algorithm
All edges cost 1	BFS
Edge costs are only 0 and 1	0-1 BFS
Positive weighted edges	Dijkstra

1. [CSES Shortest Routes I](#)

Problem Link: [Open on CSES](#)

Problem

Given a directed weighted graph, find the shortest distance from node **1** to every node.

Observation

There is no extra state. Reaching a node with a smaller distance is always better.

Why Normal Dijkstra Works

The future depends only on the current node, so:

```
State = (node)
dist[node]
```

Graph Layering Idea

No layering is required. This is the base problem before state graphs.

State Graph Construction

Each original node appears once.

Visualization

```
1 --4--> 2
|         |
2         3
v         v
3 --1--> 4
```

Process the smallest current distance first and relax outgoing edges.

Dijkstra on Graph

Priority queue stores:

```
(distance, node)
```

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const int INF = 1e18;

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(n + 1);
```

```

for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    adj[u].push_back({v, w});
}

vector<int> dist(n + 1, INF);

priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>
> pq;

dist[1] = 0;
pq.push({0, 1});

while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();

    // Ignore stale priority queue entry.
    if (d > dist[u]) continue;

    for (auto [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}

for (int i = 1; i <= n; i++) {
    cout << dist[i] << " ";
}

return 0;
}

```

Complexity

- Time Complexity: $O((N + M) \log N)$
- Space Complexity: $O(N + M)$

Pattern Learned

Normal shortest path keeps only:

`dist[node]`

Every later problem changes this into:

`dist[node][state]`

2. [LeetCode 787 — Cheapest Flights Within K Stops](#)

Problem Link: [Open on LeetCode](#)

Problem

Given flights $u \rightarrow v$ with cost, find the cheapest price from src to dst using at most K stops.

Observation

Reaching the same city with different number of used edges/stops is different because remaining stops change the future.

Why Normal Dijkstra Fails

$dist[node]$ may keep a cheaper route that used too many stops and discard a costlier route that still has enough stops left.

Graph Layering Idea

Every city is copied by edge count:

```
(city, edgesUsed)
```

For K stops, maximum edges allowed are $K + 1$.

State Graph Construction

Original edge:

```
u -> v with cost w
```

State transition:

```
(u, used) -> (v, used + 1)  
cost = w
```

Visualization

```
Layer 0: cities after 0 edges  
Layer 1: cities after 1 edge  
Layer 2: cities after 2 edges  
...
```

Dijkstra on Layered Graph

Priority queue stores:

```
(cost, node, edgesUsed)
```

C++ Code

```
class Solution {  
public:  
    int findCheapestPrice(int n, vector<vector<int>>& flights,  
                          int src, int dst, int K) {  
  
        vector<vector<pair<int,int>>> adj(n);
```

```

    for (auto &flight : flights) {
        int u = flight[0];
        int v = flight[1];
        int w = flight[2];
        adj[u].push_back({v, w});
    }

    const int INF = 1e9;
    int maxEdges = K + 1;

    // dist[node][edgesUsed]
    vector<vector<int>> dist(n, vector<int>(maxEdges + 1, INF));

    priority_queue<
        tuple<int,int,int>,
        vector<tuple<int,int,int>>,
        greater<tuple<int,int,int>>
    > pq;

    dist[src][0] = 0;
    pq.push({0, src, 0});

    while (!pq.empty()) {
        auto [cost, u, used] = pq.top();
        pq.pop();

        if (cost > dist[u][used]) continue;
        if (used == maxEdges) continue;

        for (auto [v, w] : adj[u]) {
            if (cost + w < dist[v][used + 1]) {
                dist[v][used + 1] = cost + w;
                pq.push({cost + w, v, used + 1});
            }
        }
    }

    int answer = INF;

    for (int used = 0; used <= maxEdges; used++) {
        answer = min(answer, dist[dst][used]);
    }

    return answer == INF ? -1 : answer;
}
};

```

Complexity

- Time Complexity: $O((N + M) * K * \log(NK))$
- Space Complexity: $O(NK)$

Pattern Learned

When the statement says **at most K stops**, answer is:

```
min(dist[dst][0...K+1])
```

because `dist[node][used]` means exactly `used` edges were used.

3. [LeetCode 1293 — Shortest Path in a Grid with Obstacles Elimination](#)

Problem Link: [Open on LeetCode](#)

Problem

Given a grid with empty cells and obstacles, move from `(0,0)` to `(n-1,m-1)` while removing at most `K` obstacles.

Observation

Same cell with different number of obstacles removed gives different future possibilities.

Why Normal BFS Fails

`visited[row][col]` merges states like:

```
(row, col, removed = 0)
(row, col, removed = 1)
```

But the first state is stronger because it has more removals left.

Graph Layering Idea

Each cell has `K + 1` copies:

```
(row, col, removed)
```

State Graph Construction

Move to empty cell:

```
(r, c, used) -> (nr, nc, used)
```

Move to obstacle:

```
(r, c, used) -> (nr, nc, used + 1)
```

allowed only if `used < K`.

Visualization

```
Layer 0 = no obstacle removed
Layer 1 = one obstacle removed
Layer 2 = two obstacles removed
...
```

BFS on Layered Graph

All moves cost `1`, so BFS is enough.

C++ Code

```
class Solution {
public:
    int shortestPath(vector<vector<int>>& grid, int k) {
        int n = grid.size();
        int m = grid[0].size();

        vector<vector<vector<int>>> vis(
            n,
            vector<vector<int>>(m, vector<int>(k + 1, 0))
        );

        queue<tuple<int,int,int,int>> q;
        // row, col, obstaclesRemoved, distance

        q.push({0, 0, 0, 0});
        vis[0][0][0] = 1;

        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};

        while (!q.empty()) {
            auto [r, c, used, dist] = q.front();
            q.pop();

            if (r == n - 1 && c == m - 1) {
                return dist;
            }

            for (int dir = 0; dir < 4; dir++) {
                int nr = r + dr[dir];
                int nc = c + dc[dir];

                if (nr < 0 || nr >= n || nc < 0 || nc >= m) {
                    continue;
                }

                int newUsed = used + grid[nr][nc];

                if (newUsed <= k && !vis[nr][nc][newUsed]) {
                    vis[nr][nc][newUsed] = 1;
                    q.push({nr, nc, newUsed, dist + 1});
                }
            }
        }

        return -1;
    }
};
```

Complexity

- Time Complexity: $O(N * M * K)$
- Space Complexity: $O(N * M * K)$

Pattern Learned

State graphs are not only for Dijkstra. Choose the algorithm after seeing edge weights:

```
unit edge cost -> BFS
positive weighted edges -> Dijkstra
```

4. [LeetCode 2093 — Minimum Cost to Reach City With Discounts](#)

Problem Link: [Open on LeetCode](#)

Problem

Given an undirected weighted graph and `discounts` coupons, go from city `0` to city `n-1`. Each coupon can be used once on one edge and changes the cost to `floor(w / 2)`.

Observation

Same city with different coupons used is different because remaining discounts affect future choices.

Why Normal Dijkstra Fails

A cheaper path to a city may have used more coupons, so it can be worse later.

Graph Layering Idea

```
State = (node, discountsUsed)
```

There are `discounts + 1` layers.

State Graph Construction

For every edge `u --w-- v`, from `(u, used)`:

```
Pay normally: (u, used) -> (v, used)    cost = w
Use discount: (u, used) -> (v, used + 1) cost = w / 2
```

Visualization

```
Layer 0 = no discount used
Layer 1 = one discount used
...
Layer D = D discounts used
```

Dijkstra on Layered Graph

Priority queue stores:

```
(cost, node, discountsUsed)
```

C++ Code

```
class Solution {
public:
    int minimumCost(int n, vector<vector<int>>& highways, int discounts) {
        vector<vector<pair<int,int>>> adj(n);

        for (auto &edge : highways) {
```



```

        int u = edge[0];
        int v = edge[1];
        int w = edge[2];

        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    const long long INF = 4e18;

    // dist[node][discountsUsed]
    vector<vector<long long>> dist(
        n,
        vector<long long>(discounts + 1, INF)
    );

    priority_queue<
        tuple<long long,int,int>,
        vector<tuple<long long,int,int>>,
        greater<tuple<long long,int,int>>
    > pq;

    dist[0][0] = 0;
    pq.push({0, 0, 0});

    while (!pq.empty()) {
        auto [currentCost, u, used] = pq.top();
        pq.pop();

        if (currentCost > dist[u][used]) continue;

        for (auto [v, w] : adj[u]) {
            // Move without using a discount.
            if (currentCost + w < dist[v][used]) {
                dist[v][used] = currentCost + w;
                pq.push({dist[v][used], v, used});
            }

            // Move using one discount if available.
            if (used < discounts && currentCost + w / 2 < dist[v][used + 1]) {
                dist[v][used + 1] = currentCost + w / 2;
                pq.push({dist[v][used + 1], v, used + 1});
            }
        }
    }

    long long answer = INF;

    for (int used = 0; used <= discounts; used++) {
        answer = min(answer, dist[n - 1][used]);
    }

    return answer == INF ? -1 : (int)answer;
}
};

```

Complexity

- Time Complexity: $O((N + M) * D * \log(ND))$
- Space Complexity: $O(ND)$

Pattern Learned

For every state and every edge, generate all valid next states:

```
normal edge
discount edge
```

5. [Codeforces 1725M — Moving Both Hands](#)

Problem Link: [Open on Codeforces](#)

Problem

You have a directed weighted graph. For every $p = 2 \dots N$, left hand starts at node 1, right hand starts at node p , and one hand moves at a time. Find the minimum time until both hands meet at the same vertex.

Observation

If both hands meet at node x , total cost is:

```
distance(1, x) + distance(p, x)
```

We need this for every p .

Why Running Dijkstra From Every p Fails

Running Dijkstra separately for every starting p is too slow:

```
 $O(N * M \log N)$ 
```

Graph Layering Idea

Use two layers:

```
Layer 0 = original graph
Layer 1 = reversed graph
```

Move from layer 0 to layer 1 with cost 0.

State Graph Construction

For original edge:

```
u -> v with weight w
```

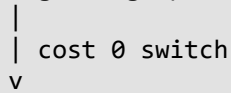
Add:

```
u0 -> v0 with cost w
v1 -> u1 with cost w
u0 -> u1 with cost 0
```

The zero edge means: choose the meeting point and then continue in reversed graph.

Visualization

Layer 0: original graph from node 1 to meeting point



Layer 1: reversed graph from meeting point to p

Dijkstra on Layered Graph

Run one Dijkstra from node 1 in layer 0.

Answer for node p is distance to p in layer 1.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const long long INF = 4e18;

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(2 * n + 1);

    auto normal = [&](int node) {
        return node;
    };

    auto reversed = [&](int node) {
        return node + n;
    };

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;

        // Layer 0: original edge.
        adj[normal(u)].push_back({normal(v), w});

        // Layer 1: reversed edge.
        adj[reversed(v)].push_back({reversed(u), w});
    }

    for (int node = 1; node <= n; node++) {
        // Choose meeting point, then enter reversed layer.
        adj[normal(node)].push_back({reversed(node), 0});
    }
}
```

```

vector<int> dist(2 * n + 1, INF);

priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>
> pq;

dist[normal(1)] = 0;
pq.push({0, normal(1)});

while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();

    if (d > dist[u]) continue;

    for (auto [v, w] : adj[u]) {
        if (d + w < dist[v]) {
            dist[v] = d + w;
            pq.push({dist[v], v});
        }
    }
}

for (int p = 2; p <= n; p++) {
    int answer = dist[reversed(p)];
    cout << (answer >= INF / 2 ? -1 : answer) << " ";
}

return 0;
}

```

Complexity

- Time Complexity: $O((N + M) \log N)$
- Space Complexity: $O(N + M)$

Pattern Learned

When answer looks like:

$\text{dist}(A, x) + \text{dist}(B, x)$

try reversing the graph so "distance to x" becomes "distance from x".

6. [CSES Flight Discount](#)

Problem Link: [Open on CSES](#)

Problem

Given directed weighted flights from 1 to N, use exactly one discount coupon on one flight, changing cost w to $\text{floor}(w / 2)$. Find minimum cost.

Observation

At a city, coupon unused and coupon used are different states.

Why Normal Dijkstra Fails

A route that is currently cheaper may already have used the coupon, while a slightly costlier route may save the coupon for a large edge later.

Graph Layering Idea

State = (node, couponUsed)

Two layers:

0 = coupon not used

1 = coupon used

State Graph Construction

For edge $u \rightarrow v$ with weight w :

```
(u,0) -> (v,0) cost w
(u,0) -> (v,1) cost w/2
(u,1) -> (v,1) cost w
```

Visualization

Layer 0: coupon available

Layer 1: coupon already used

Diagonal edge = use coupon

Dijkstra on Layered Graph

Answer is:

```
dist[n][1]
```

because coupon must be used.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const long long INF = 4e18;

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(n + 1);

    for (int i = 0; i < m; i++) {
        int u, v, w;
```

```

    cin >> u >> v >> w;
    adj[u].push_back({v, w});
}

vector<vector<int>> dist(n + 1, vector<int>(2, INF));

priority_queue<
    tuple<int,int,int>,
    vector<tuple<int,int,int>>,
    greater<tuple<int,int,int>>
> pq;

dist[1][0] = 0;
pq.push({0, 1, 0});

while (!pq.empty()) {
    auto [d, u, used] = pq.top();
    pq.pop();

    if (d > dist[u][used]) continue;

    for (auto [v, w] : adj[u]) {
        // Do not use coupon on this edge.
        if (d + w < dist[v][used]) {
            dist[v][used] = d + w;
            pq.push({dist[v][used], v, used});
        }

        // Use coupon on this edge.
        if (!used && d + w / 2 < dist[v][1]) {
            dist[v][1] = d + w / 2;
            pq.push({dist[v][1], v, 1});
        }
    }
}

cout << dist[n][1] << "\n";

return 0;
}

```

Complexity

- Time Complexity: $O((N + M) \log N)$
- Space Complexity: $O(N + M)$

Pattern Learned

One coupon problems can be solved by:

`dist[node][couponUsed]`

or by the two-Dijkstra trick, but layered Dijkstra generalizes better.

7. [AtCoder ABC277E — Crystal Switches](#)

Problem Link: [Open on AtCoder](#)

Problem

You have an undirected graph where each edge is usable only in one switch state. Some nodes have switches that toggle the state. Find the shortest path from **1** to **N**.

Observation

At the same node, switch **ON** and switch **OFF** give different available edges.

Why Normal BFS Fails

`visited[node]` loses whether the current world mode is **0** or **1**.

Graph Layering Idea

State = (node, switchState)

Two layers:

0 = OFF

1 = ON

State Graph Construction

Edge with type **a** exists only in layer **a**.

If node has switch:

(node,0) <-> (node,1) with cost 0

Road movement costs **1**.

Visualization

Layer 0: edges usable when switch state is 0

Layer 1: edges usable when switch state is 1

Switch nodes connect both layers with cost 0

0-1 BFS on Layered Graph

Road edges cost **1**, switch edges cost **0**, so use 0-1 BFS.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k;
    cin >> n >> m >> k;
```

```

vector<vector<int>> adj[2];
adj[0].assign(n + 1, {});
adj[1].assign(n + 1, {});

for (int i = 0; i < m; i++) {
    int u, v, a;
    cin >> u >> v >> a;

    // Edge is usable only when current state equals a.
    adj[a][u].push_back(v);
    adj[a][v].push_back(u);
}

vector<int> hasSwitch(n + 1, 0);

for (int i = 0; i < k; i++) {
    int node;
    cin >> node;
    hasSwitch[node] = 1;
}

vector<vector<int>> dist(n + 1, vector<int>(2, INF));
deque<pair<int,int>> dq;

// At start, switch state is ON = 1.
dist[1][1] = 0;
dq.push_front({1, 1});

while (!dq.empty()) {
    auto [u, state] = dq.front();
    dq.pop_front();

    // Toggle switch with cost 0.
    if (hasSwitch[u] && dist[u][state] < dist[u][state ^ 1]) {
        dist[u][state ^ 1] = dist[u][state];
        dq.push_front({u, state ^ 1});
    }

    // Travel using edges available in current state.
    for (int v : adj[state][u]) {
        if (dist[u][state] + 1 < dist[v][state]) {
            dist[v][state] = dist[u][state] + 1;
            dq.push_back({v, state});
        }
    }
}

int answer = min(dist[n][0], dist[n][1]);
cout << (answer == INF ? -1 : answer) << "\n";

return 0;
}

```


Complexity

- Time Complexity: $O(N + M + K)$
- Space Complexity: $O(N + M)$

Pattern Learned

When a problem says switch/toggle/mode, think:

```
(node, mode)
```

8. [AtCoder ABC395E — Flip Edge](#)

Problem Link: [Open on AtCoder](#)

Problem

Given a directed graph. Moving along an edge costs 1 . You may flip the entire graph any number of times, each flip costs X . Find minimum cost from 1 to N .

Observation

At the same node, being in original graph mode or reversed graph mode gives different outgoing edges.

Why Normal Dijkstra Fails

$dist[node]$ does not tell which edge orientation is currently active.

Graph Layering Idea

```
State = (node, graphMode)
```

Two layers:

```
0 = original graph  
1 = reversed graph
```

State Graph Construction

For directed edge $u \rightarrow v$:

```
u0 -> v0 cost 1  
v1 -> u1 cost 1
```

For every node:

```
u0 <-> u1 cost X
```

Visualization

```
Layer 0: original direction  
Layer 1: reversed direction  
Vertical edges: flip whole graph, cost X
```

Dijkstra on Layered Graph

Run Dijkstra from $(1, 0)$.

Answer:

```
min(dist[N][0], dist[N][1])
```

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const long long INF = 4e18;

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, x;
    cin >> n >> m >> x;

    vector<vector<pair<int,int>>> adj(2 * n + 2);

    auto normal = [&](int node) {
        return 2 * node;
    };

    auto reversed = [&](int node) {
        return 2 * node + 1;
    };

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;

        // Original graph layer.
        adj[normal(u)].push_back({normal(v), 1});

        // Reversed graph layer.
        adj[reversed(v)].push_back({reversed(u), 1});
    }

    for (int node = 1; node <= n; node++) {
        // Flip graph orientation.
        adj[normal(node)].push_back({reversed(node), x});
        adj[reversed(node)].push_back({normal(node), x});
    }

    vector<int> dist(2 * n + 2, INF);

    priority_queue<
        pair<int,int>,
        vector<pair<int,int>>,
        greater<pair<int,int>>
    > pq;
```

```

dist[normal(1)] = 0;
pq.push({0, normal(1)});

while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();

    if (d > dist[u]) continue;

    for (auto [v, w] : adj[u]) {
        if (d + w < dist[v]) {
            dist[v] = d + w;
            pq.push({dist[v], v});
        }
    }
}

int answer = min(dist[normal(n)], dist[reversed(n)]);
cout << (answer >= INF / 2 ? -1 : answer) << "\n";

return 0;
}

```

Complexity

- Time Complexity: $O((N + M) \log N)$
- Space Complexity: $O(N + M)$

Pattern Learned

If the entire graph can flip between two versions, build two layers and connect same nodes across layers.

9. [LeetCode 2714 — Find Shortest Path with K Hops](#)

Problem Link: [Open on LeetCode](#)

Problem

Given an undirected weighted graph, go from **source** to **destination**. You may use at most **K** hops, where one hop makes an edge cost 0.

Observation

Same node with different number of hops used gives different future choices.

Why Normal Dijkstra Fails

A path that is cheaper now may have used too many free hops, so it can be worse later.

Graph Layering Idea

State = (node, hopsUsed)

There are $K + 1$ layers.

State Graph Construction

For edge $u \xrightarrow{w} v$:

```
normal move: (u, used) -> (v, used)    cost w
free hop:    (u, used) -> (v, used + 1) cost 0
```

Visualization

```
Layer 0 = no hop used
Layer 1 = one hop used
...
Layer K = K hops used
```

Dijkstra on Layered Graph

Answer:

```
min(dist[destination][0...K])
```

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const long long INF = 4e18;

void solve() {
    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(n + 1);

    for (int i = 0; i < m; i++) {
        int u, v, wt;
        cin >> u >> v >> wt;

        adj[u].push_back({v, wt});
        adj[v].push_back({u, wt});
    }

    int source, destination, k;
    cin >> source >> destination >> k;

    vector<vector<int>> dist(n + 1, vector<int>(k + 1, INF));

    priority_queue<
        tuple<int,int,int>,
        vector<tuple<int,int,int>>,
        greater<tuple<int,int,int>>
    > pq;

    dist[source][0] = 0;
    pq.push({0, source, 0});

    while (!pq.empty()) {
```

```

    auto [currentDist, u, used] = pq.top();
    pq.pop();

    if (currentDist > dist[u][used]) continue;

    for (auto [v, w] : adj[u]) {
        // Use one free hop.
        if (used < k && currentDist < dist[v][used + 1]) {
            dist[v][used + 1] = currentDist;
            pq.push({dist[v][used + 1], v, used + 1});
        }

        // Pay normal edge cost.
        if (currentDist + w < dist[v][used]) {
            dist[v][used] = currentDist + w;
            pq.push({dist[v][used], v, used});
        }
    }

    int answer = INF;

    for (int used = 0; used <= k; used++) {
        answer = min(answer, dist[destination][used]);
    }

    cout << (answer == INF ? -1 : answer) << "\n";
}

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
    return 0;
}

```

Complexity

- Time Complexity: $O((N + M) * K * \log(NK))$
- Space Complexity: $O(NK)$

Pattern Learned

This is the same as discounts/coupons, except special transition cost is 0.

10. [Codeforces 1473E — Minimum Path](#)

Problem Link: [Open on Codeforces](#)

Problem

For every node, find minimum path cost from 1, where path weight can be seen as:

```
sum of edges - one chosen edge + one chosen edge
```

Equivalently:

```
use one free operation  
use one double operation
```

Both operations may be applied to the same edge.

Observation

You do not explicitly track max edge and min edge.

Dijkstra chooses where to use:

```
free operation  
double operation
```

Why Normal Dijkstra Fails

At the same node, four situations are different:

```
free unused, double unused  
free used, double unused  
free unused, double used  
free used, double used
```

Graph Layering Idea

```
State = (node, freeUsed, doubleUsed)
```

Four layers.

State Graph Construction

For edge of weight w , from current state:

```
Use nothing: cost  $w$   
Use free:    cost  $0$   
Use double:  cost  $2w$   
Use both:   cost  $w$ 
```

Visualization

```
00 = nothing used  
10 = free used  
01 = double used  
11 = both used
```

Dijkstra on Layered Graph

Answer for node i :

```
dist[i][1][1]
```

C++ Code

```
#include <bits/stdc++.h>  
using namespace std;
```

```

#define int long long
const long long INF = 4e18;

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(n + 1);

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;

        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    // dist[node][freeUsed][doubleUsed]
    vector<vector<vector<int>>> dist(
        n + 1,
        vector<vector<int>>(2, vector<int>(2, INF))
    );

    priority_queue<
        tuple<int,int,int,int>,
        vector<tuple<int,int,int,int>>,
        greater<tuple<int,int,int,int>>
    > pq;

    dist[1][0][0] = 0;
    pq.push({0, 1, 0, 0});

    while (!pq.empty()) {
        auto [d, u, freeUsed, doubleUsed] = pq.top();
        pq.pop();

        if (d != dist[u][freeUsed][doubleUsed]) continue;

        for (auto [v, w] : adj[u]) {
            // 1. Use nothing.
            if (d + w < dist[v][freeUsed][doubleUsed]) {
                dist[v][freeUsed][doubleUsed] = d + w;
                pq.push({d + w, v, freeUsed, doubleUsed});
            }

            // 2. Use free operation.
            if (!freeUsed && d < dist[v][1][doubleUsed]) {
                dist[v][1][doubleUsed] = d;
                pq.push({d, v, 1, doubleUsed});
            }

            // 3. Use double operation.

```

```

        if (!doubleUsed && d + 2 * w < dist[v][freeUsed][1]) {
            dist[v][freeUsed][1] = d + 2 * w;
            pq.push({d + 2 * w, v, freeUsed, 1});
        }

        // 4. Use both operations on this same edge.
        if (!freeUsed && !doubleUsed && d + w < dist[v][1][1]) {
            dist[v][1][1] = d + w;
            pq.push({d + w, v, 1, 1});
        }
    }
}

for (int node = 2; node <= n; node++) {
    cout << dist[node][1][1] << " ";
}

return 0;
}

```

Complexity

- Time Complexity: $O((N + M) \log N)$
- Space Complexity: $O(N)$

Pattern Learned

Multiple independent one-time operations become bits/layers:

(node, mask)

11. [Codeforces 1486E — Paired Payment](#)

Problem Link: [Open on Codeforces](#)

Problem

Given an undirected weighted graph. Path edges are grouped into pairs. A pair (a, b) contributes:

$(a + b)^2$

If one edge is unpaired, it waits and adds no cost until the next edge arrives.

Observation

Current node is not enough. The previous unpaired edge weight affects the cost of the next edge.

Why Normal Dijkstra Fails

At the same node:

```

lastEdgeWeight = 4
lastEdgeWeight = 9

```

are different states because the next edge cost changes.

Graph Layering Idea

State = (node, lastEdgeWeight)

lastEdgeWeight = 0 means no pending edge.

State Graph Construction

If no pending edge:

```
(node, 0) -> (next, w)
cost = 0
```

If pending edge exists:

```
(node, last) -> (next, 0)
cost = (last + w)^2
```

Visualization

```
(1,0) --edge 2, cost 0--> (2,2)
(2,2) --edge 3, cost 25--> (3,0)
```

Dijkstra on Layered Graph

Priority queue stores:

(cost, node, lastEdgeWeight)

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const long long INF = 4e18;

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(n + 1);

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;

        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    // Edge weights are small, so lastEdgeWeight ranges from 0 to 50.
    vector<vector<int>> dist(n + 1, vector<int>(51, INF));

    priority_queue<
```

```

        tuple<int,int,int>,
        vector<tuple<int,int,int>>,
        greater<tuple<int,int,int>>
    > pq;

    dist[1][0] = 0;
    pq.push({0, 1, 0});

    while (!pq.empty()) {
        auto [d, u, last] = pq.top();
        pq.pop();

        if (d != dist[u][last]) continue;

        for (auto [v, w] : adj[u]) {
            if (last == 0) {
                // Start a new pair; no cost is added yet.
                if (d < dist[v][w]) {
                    dist[v][w] = d;
                    pq.push({d, v, w});
                }
            } else {
                // Complete the pair.
                int newCost = d + (last + w) * (last + w);

                if (newCost < dist[v][0]) {
                    dist[v][0] = newCost;
                    pq.push({newCost, v, 0});
                }
            }
        }
    }

    for (int node = 2; node <= n; node++) {
        if (dist[node][0] == INF) {
            cout << -1 << " ";
        } else {
            cout << dist[node][0] << " ";
        }
    }

    return 0;
}

```

Complexity

- Time Complexity: $O(51 * M * \log(51N))$
- Space Complexity: $O(51N)$

Pattern Learned

When next transition depends on previous edge/move, use:

```
(node, previousInformation)
```

Pending Roadmap Items From The Chat

These were mentioned in the roadmap but were not fully completed in the linked chat:

Problem	Status
CodeChef REVERSE	Mentioned as practice; full notes not present in linked chat
AtCoder ABC164E	Mentioned as future problem
Codeforces 1433G	Mentioned as future problem

If needed, prepare these next in the same exact format.